

Semaine 2 | Module B

HTML5 — Structure & sémantique

Anne Terrier — Développement web, classe passerelle

Cours 3h

TP 4h

Fil rouge MiniForum

Objectifs de la semaine

- Écrire une page HTML5 valide, correctement imbriquée
- Connaître le squelette d'un document et le rôle du `<head>`
- Choisir la balise qui correspond au *sens* du contenu, pas à son apparence
- Structurer le texte : hiérarchie des titres, paragraphes, listes, tableaux
- Créer des liens et insérer des images avec des chemins corrects
- Valider une page sur validator.w3.org et savoir lire les erreurs signalées

Où en sommes-nous ?

La semaine 1 a posé le décor : le modèle client / serveur, les trois langages du navigateur, et l'environnement de travail. Cette semaine, on entre dans le premier de ces langages — **HTML**, celui qui décrit la structure. Le style viendra en semaine 4, l'interactivité en semaine 6. Pour l'instant, la page sera laide, et ce n'est pas un problème.

Comment se servir de ce polycopié cette semaine

Le cours ne se déroule pas en exposé mais en **quatre défis**, sur des fichiers jetables du dossier `labo-web/s02`. Pendant chaque défi, ce document est ta ressource : la diapositive projetée t'indique les sections à ouvrir.

Défi	Sections ouvertes	Ce qu'on cherche
1 — la recette	§1.1, §1.2	pourquoi la page ne ressemble pas au fichier
2 — le club de randonnée	§1, §2	pourquoi une page se laisse lire, et l'autre non
3 — les chemins cassés	§5.1, §5.4	d'où se lit un chemin
4 — le validateur	tout sauf §7	ce que le navigateur cachait

Une section n'est jamais indiquée pendant le défi dont elle est la réponse : **§1.3** et **§1.4** au défi 1, **§3** et **§4.2** au défi 2, **§5.2** au défi 3, **§7** au défi 4. Elles sont nommées en classe une fois la recherche terminée — puis relues ici.

Enfin, plusieurs points ne sont pas traités en cours faute de temps et te reviennent en lecture personnelle : le détail du `<head>` (§2.1), les éléments en bloc et en ligne (§4.3), l'attribut `id` (§5.3), les listes ordonnées et les tableaux (§6). Ils sont au quiz.

1 Le HTML, un langage de balisage

HTML signifie *HyperText Markup Language* : langage de balisage hypertexte. Une page web est un simple fichier texte portant l'extension `.html`, que le navigateur lit et interprète pour construire l'affichage.

Définition — Langage de balisage

Un **langage de balisage** sert à *décrire* un contenu, pas à effectuer des calculs. On entoure chaque morceau de texte d'une **balise** qui indique ce qu'il représente : un titre, un paragraphe, un lien, une image. . .

HTML n'est donc **pas un langage de programmation** : il n'a ni variables, ni conditions, ni boucles. Ces notions arriveront avec JavaScript, en semaine 6.

Anatomie d'une balise

La plupart des éléments HTML fonctionnent par **paires** : une balise ouvrante et une balise fermante encadrent un contenu.

`<p> Bonjour tout le monde </p>`

↑ ↑ ↑

Balise ouvrante Contenu Balise fermante

L'ensemble — balise ouvrante, contenu, balise fermante — forme un **élément**. La balise fermante se distingue par sa barre oblique : `</p>`.

Certains éléments n'encadrent aucun contenu et s'écrivent donc **seuls**. On les dit **auto-fermants** : `<img>` (une image), `<br>` (un retour à la ligne forcé), `<meta>`, `<link>`, `<input>`.

Les attributs

Une balise peut porter des **attributs**, qui précisent son comportement. Un attribut s'écrit dans la balise **ouvrante**, sous la forme `nom="valeur"`.

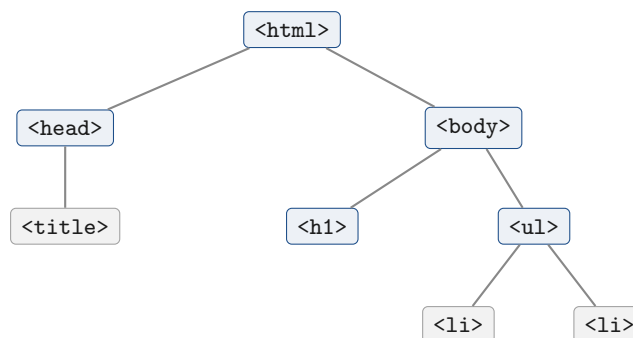
```
1 <a href="https://www.hepia.ch" target="_blank">Site de la hepia</a>
```

Extrait 1 – Une balise, deux attributs

Ici, `href` indique la destination du lien et `target` précise qu'il doit s'ouvrir dans un nouvel onglet. Une balise fermante, elle, ne porte **jamais** d'attribut.

L'imbrication : un document est un arbre

Les éléments se placent les uns **dans** les autres. Cette imbrication n'est pas décorative : elle construit une structure arborescente que le navigateur, les moteurs de recherche et — plus tard — ton code JavaScript vont parcourir.



La règle d'imbrication : premier ouvert, dernier fermé

Les balises se ferment dans l'ordre inverse de leur ouverture, comme des parenthèses. Elles ne peuvent pas se chevaucher.

OUI `<p>Un <strong>mot</strong> important</p>`

NON `<p>Un <strong>mot</p></strong> important`

Indente ton code : un niveau d'indentation par niveau d'imbrication. C'est ce qui rend les erreurs visibles à l'œil nu.

Espaces, retours à la ligne et commentaires**Le HTML ignore ta mise en forme du texte**

Dans un fichier HTML, plusieurs espaces consécutifs, les tabulations et les retours à la ligne sont **réduits à un seul espace** à l'affichage. Écrire trois lignes vides ne crée aucun blanc dans la page ; appuyer sur Entrée ne crée pas de nouveau paragraphe.

C'est le **balisage** qui produit la mise en forme : un nouveau paragraphe s'obtient avec un nouveau `<p>`, les espacements avec du CSS (semaine 4).

```
1 <!-- Liste des sujets du forum - à remplacer par du contenu
2      généré en JavaScript à partir de la semaine 7 -->
3 <ul>
4   <li>Bienvenue sur le MiniForum</li>
5 </ul>
```

Extrait 2 – Un commentaire n'est pas affiché dans la page

Le commentaire n'est pas un secret

Un commentaire HTML est invisible *dans la page*, mais parfaitement lisible par n'importe qui via *Afficher le code source* (Ctrl+U). Il sert à expliquer ton code, jamais à y cacher une information.

2 La structure d'un document HTML5

Toute page HTML5 repose sur le même squelette. Il est à connaître par cœur :

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>MiniForum</title>
7   <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
10  <!-- Tout le contenu visible de la page -->
11 </body>
12 </html>
```

Extrait 3 – Squelette minimal d'une page HTML5

Élément	Rôle
<code><!DOCTYPE html></code>	Annonce au navigateur qu'il s'agit d'un document HTML5. Sans lui, le navigateur bascule dans un mode de compatibilité imprévisible
<code><html lang="fr"></code>	La racine du document ; <code>lang</code> indique la langue du contenu
<code><head></code>	Métadonnées invisibles : encodage, titre d'onglet, feuilles de style
<code><body></code>	Tout le contenu visible de la page

Ce que contient le `<head>`

Balise	Ce qu'elle fait
<code><meta charset="UTF-8"></code>	Définit l'encodage des caractères. Toujours UTF-8 : sans cette ligne, les accents s'affichent en caractères illisibles
<code><meta name="viewport"></code>	Indique au téléphone d'afficher la page à sa taille réelle. Indispensable pour le responsive (semaine 5)
<code><title></code>	Le texte affiché dans l'onglet, dans les favoris et dans les résultats de recherche
<code><meta name="description"></code>	Le résumé affiché sous le titre dans les résultats des moteurs de recherche
<code><link rel="stylesheet"></code>	Rattache une feuille de style CSS externe

`<head>` et `<body>` : ne pas confondre

Tout ce qui doit être **vu** par le visiteur va dans le `<body>`. Le `<head>` ne contient que des informations *sur* la page. Un titre écrit dans le `<head>` n'apparaîtra jamais dans la page — c'est l'erreur de débutant la plus courante de la semaine.

Gagner du temps dans VS Code

Dans un fichier `.html` vide, tape ! puis Tab : VS Code génère l'intégralité du squelette ci-dessus. Utilise ce raccourci — mais assure-toi de savoir le réécrire à la main, car c'est ce qui sera demandé au contrôle écrit de la semaine 12.

3 Les éléments sémantiques

Pendant longtemps, les pages web ont été construites avec des `<div>` partout : des boîtes sans signification, distinguées uniquement par leur nom de classe. HTML5 a introduit des balises **sémantiques**, qui décrivent le **rôle** de chaque zone.

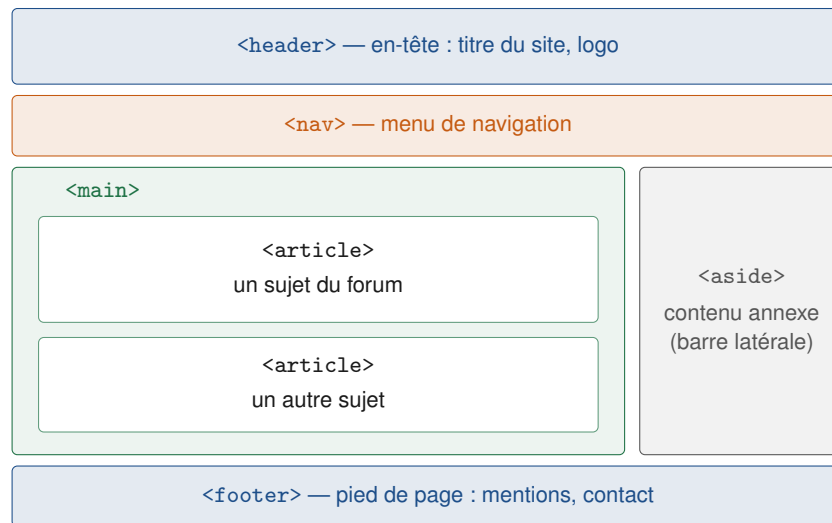
Définition — Sémantique

Une balise **sémantique** indique ce que le contenu *est*, pas la façon dont il doit s'afficher. `<nav>` signifie « ceci est une zone de navigation », indépendamment de sa couleur ou de sa position.

Trois publics en tirent parti :

- les **humains** qui relisent le code — toi, dans trois semaines, ou un collègue ;
- les **moteurs de recherche**, qui comprennent où se trouve le contenu principal ;
- les **lecteurs d'écran**, qui permettent aux personnes malvoyantes de sauter directement à la navigation ou au contenu. C'est le sujet de la semaine 3.

La carte d'une page



Balise	Quand l'utiliser
<code><header></code>	En-tête de la page ou d'une section : titre, logo, date de publication
<code><nav></code>	Un ensemble de liens de navigation. Réservé aux menus principaux, pas au moindre lien isolé
<code><main></code>	Le contenu principal. Un seul par page , et il ne contient ni en-tête ni menu
<code><section></code>	Un regroupement thématique, normalement introduit par un titre
<code><article></code>	Un contenu autonome : un message du forum, un billet de blog
<code><aside></code>	Un contenu annexe, secondaire par rapport au contenu principal
<code><footer></code>	Pied de page de la page ou d'une section

`<section>` ou `<article>` ?

Le test : est-ce que ce contenu garderait du sens **sorti de la page**, publié seul ailleurs ?
 Si oui, c'est un `<article>` — un message de forum que l'on pourrait citer sur un autre site. Si non, c'est une `<section>` — « Sujets récents » n'a de sens que dans le forum qui la contient.

Ce qui reste du `<div>`

Les `<div>` et `<span>` n'ont pas disparu ; ils ont simplement cessé d'être le choix par défaut.

Balise	Usage légitime
<code><div></code>	Regrouper des éléments uniquement pour les mettre en forme, sans aucune signification à exprimer
<code></code>	Idem, mais à l'intérieur d'une ligne de texte (pour colorer un mot, par exemple)

Une balise sémantique ne change (presque) rien à l'affichage

Remplacer un `<div>` par un `<main>` ne modifie pas visiblement la page. C'est déroutant au début : on a l'impression que le choix n'a pas d'importance.

Il en a une, mais elle est **invisible** — elle se joue du côté des lecteurs d'écran, des moteurs de recherche et de la lisibilité du code. C'est aussi pour cela que le TP de cette semaine se fait **sans CSS** : rien ne doit venir masquer la structure.

4 Structurer le texte

Titres et paragraphes

```

1 <h1>MiniForum</h1>
2 <h2>Sujets récents</h2>
3 <h3>Bienvenue sur le forum</h3>
4
5 <p>Un paragraphe de texte, dans lequel on peut signaler du
6   <strong>contenu important</strong> ou mettre un mot en
7   <em>emphase</em>.</p>

```

Extrait 4 – Titres et texte

Balise	Sens
<h1>...<h6>	Six niveaux de titres, du plus important au plus fin
<p>	Un paragraphe de texte
	Contenu d' importance forte (s'affiche en gras)
	<i>Emphase</i> , mot sur lequel on insiste (s'affiche en italique)
 	Retour à la ligne à l'intérieur d'un paragraphe (une adresse postale, un vers)
<blockquote>	Une citation longue, extraite d'une autre source

 plutôt que , **plutôt que** <i>

 et <i> décrivent une *apparence* (gras, italique) ; et décrivent un *sens*. Le rendu est identique, mais un lecteur d'écran change d'intonation sur un et ignore un .

La règle générale du HTML est là tout entière : **choisir la balise selon le sens**, et laisser le CSS s'occuper de l'apparence.

La hiérarchie des titres

Les titres ne sont pas un outil de mise en forme : ils construisent le **plan** du document, exactement comme les niveaux de titres d'un rapport.

Plan correct	Plan incorrect
h1 MiniForum	h1 MiniForum
h2 Sujets récents	h3 Sujets récents (saut de niveau)
h3 Bienvenue	h2 Bienvenue
h3 Installer Node.js	h1 Nouveau sujet (deuxième h1)
h2 Nouveau sujet	

Deux règles suffisent : **un seul <h1> par page**, et **on ne saute pas de niveau** en descendant. Choisir un <h3> parce qu'il « fait la bonne taille » est une erreur : la taille se règle en CSS.

Éléments en bloc et éléments en ligne

Définition — Bloc et en ligne

Un élément **en bloc** occupe toute la largeur disponible et provoque un retour à la ligne : <p>, <h1>, , <section>, <div>.

Un élément **en ligne** se place dans le fil du texte, sans le couper : <a>, , , , .

Cette distinction explique dès maintenant pourquoi deux paragraphes se suivent verticalement alors que deux liens restent côte à côte. Elle deviendra centrale en semaine 4, avec le modèle de boîte du CSS.

5 Liens et images

Le lien hypertexte

Le lien est ce qui fait du web un web. Il se crée avec la balise `<a>` (*anchor*) et son attribut `href`.

```

1 <!-- Lien externe : URL complète -->
2 <a href="https://www.hepia.ch">Site de la hepia</a>
3
4 <!-- Lien interne au site : vers un autre fichier -->
5 <a href="sujet.html">Voir le sujet</a>
6
7 <!-- Lien vers un élément de la page courante -->
8 <a href="#nouveau-sujet">Créer un sujet</a>
9
10 <!-- Lien ouvrant un nouvel onglet -->
11 <a href="https://developer.mozilla.org" target="_blank">MDN</a>

```

Extrait 5 – Quatre types de liens

Écrire un bon intitulé de lien

Le texte du lien doit décrire sa destination **hors contexte**. Les lecteurs d'écran proposent en effet de lister tous les liens d'une page : une liste de « cliquez ici » répétés n'informe personne.

OUI `<a href="sujet.html">Lire le sujet complet</a>`

NON `<a href="sujet.html">cliquez ici</a>`

Chemins relatifs et chemins absolus

C'est la source d'erreur numéro un du TP : l'image ne s'affiche pas, le lien ne mène nulle part. Presque toujours, le chemin est en cause.

```

1 miniforum/
2 |-- index.html
3 |-- sujet.html
4 |-- style.css
5 \-- images/
6     |-- logo.png

```

Extrait 6 – Arborescence du projet MiniForum

Écriture	Signification
<code>sujet.html</code>	Un fichier du même dossier
<code>images/logo.png</code>	Le fichier <code>logo.png</code> , dans le sous-dossier <code>images</code>
<code>../style.css</code>	Remonter d'un dossier, puis prendre <code>style.css</code>
<code>https://...</code>	Une adresse absolue , sur un autre serveur

Trois pièges classiques

- **La casse compte.** `Logo.png` et `logo.png` sont deux fichiers différents pour un serveur Linux — même si Windows ne fait pas la différence. Ta page marchera chez toi et cassera en ligne.
- **Pas d'espaces ni d'accents** dans les noms de fichiers : écris `photo-profil.png`, jamais `photo profil.png` ni `été.jpg`.
- **Jamais de chemin C:/Users/...** : ce chemin n'existe que sur ta machine. Tous les chemins d'un site sont relatifs au projet.

L'attribut id

L'attribut `id` donne un **nom unique** à un élément de la page. Le lien `href="#nouveau-sujet"` vu plus haut ne pointe pas vers un fichier, mais vers l'élément qui porte cet identifiant.

```
1 <a href="#nouveau-sujet">Créer un sujet</a>
2
3 <!-- ... plus bas dans la page ... -->
4 <section id="nouveau-sujet">
5   <h2>Créer un nouveau sujet</h2>
6 </section>
```

Extrait 7 – Un id sert de cible à un lien interne

Règle d'or — un id est unique dans la page

Deux éléments ne peuvent **jamais** partager le même `id` : c'est ce qui garantit qu'un `id` désigne un élément, et un seul. Le validateur W3C signale d'ailleurs les doublons comme des erreurs.

L'id, un point d'accroche pour toute l'année

L'id ne sert pas qu'aux liens internes. C'est le point d'accroche universel vers un élément : il reliera un `<label>` à son champ (semaine 3), permettra de cibler un élément en **CSS** (semaine 4), et de le manipuler en **JavaScript** (semaine 7). Souviens-toi du premier exemple de la semaine 1 : `document.getElementById('btn')` allait chercher le bouton par son `id`.

Les images

```
1 
2
3 <figure>
4   
6   <figcaption>Organisation des pages du MiniForum</figcaption>
7 </figure>
```

Extrait 8 – Insérer une image, avec ou sans légende

Attribut	Rôle
<code>src</code>	Le chemin vers le fichier image (<i>source</i>)
<code>alt</code>	Le texte alternatif décrivant l'image
<code>width/height</code>	Dimensions en pixels; les renseigner évite que la page « saute » pendant le chargement

L'attribut alt n'est pas optionnel

Le texte `alt` est lu à voix haute par les lecteurs d'écran et s'affiche si l'image ne charge pas. Le validateur W3C signale son absence comme une erreur.

OUI `alt="Logo du MiniForum"`

NON `alt="image"` ou `alt="photo de logo.png"`

Cas particulier : une image purement décorative, qui n'apporte aucune information, prend un **alt vide** — `alt=""` — pour que le lecteur d'écran l'ignore au lieu d'annoncer le nom du fichier. On approfondira en semaine 3.

6 Listes et tableaux

Les listes

```

1 <!-- Liste à puces : l'ordre est indifférent -->
2 <ul>
3   <li>HTML</li>
4   <li>CSS</li>
5   <li>JavaScript</li>
6 </ul>
7
8 <!-- Liste numérotée : l'ordre compte -->
9 <ol>
10  <li>Ouvrir le terminal</li>
11  <li>Taper la commande</li>
12  <li>Valider</li>
13 </ol>

```

Extrait 9 – Liste non ordonnée et liste ordonnée

Un vit toujours dans un ou un

Rien d'autre qu'un ne peut être enfant direct d'un — pas de <p>, pas de <div>. Ce contenu doit être placé à l'intérieur du . Le validateur est intraitable sur ce point.

Une liste peut en revanche en contenir une autre : la sous-liste se place **dans** le parent, avant sa fermeture.

Un menu de navigation est une liste de liens

Les barres de navigation s'écrivent presque toujours ainsi : un <nav> contenant un de liens. Cela paraît étrange tant qu'on pense en termes d'apparence — mais un menu *est* bien une liste, et le CSS se chargera plus tard de l'afficher horizontalement.

Les tableaux

Un tableau se construit ligne par ligne (<tr>), avec des cellules d'en-tête (<th>) et des cellules de données (<td>).

```

1 <table>
2   <caption>Derniers sujets publiés</caption>
3   <thead>
4     <tr>
5       <th>Sujet</th>
6       <th>Auteur</th>
7     </tr>
8   </thead>
9   <tbody>
10    <tr>
11      <td>Bienvenue</td>
12      <td>Alice</td>
13    </tr>
14    <tr>
15      <td>Installer Node.js</td>
16      <td>Bob</td>
17    </tr>
18  </tbody>
19 </table>

```

Extrait 10 – Un tableau complet

Balise	Rôle
<caption>	Le titre du tableau ; première balise à l'intérieur de <table>
<thead> / <tbody>	Séparent la ligne d'en-tête du corps des données
<tr>	Une ligne (<i>table row</i>)
<th>	Une cellule d'en-tête (<i>table header</i>)
<td>	Une cellule de données (<i>table data</i>)

Fusionner des cellules

```

1 <tr>
2   <th colspan="2">Coordonnées</th>
3 </tr>
4 <tr>
5   <td>E-mail</td>
6   <td>alice@exemple.ch</td>
7 </tr>

```

Extrait 11 – Fusion horizontale avec colspan

La cellule « Coordonnées » occupe la largeur de **deux** colonnes grâce à `colspan="2"`. L'attribut `rowspan="2"` produit le même effet sur la hauteur, en fusionnant deux lignes.

Un tableau sert à présenter des données, jamais à mettre en page

Un <table> présente des **données tabulaires** : un horaire, un relevé de notes, une comparaison. Il ne doit **jamais** servir à positionner des éléments dans la page — c'est le rôle du CSS, vu en semaines 4 et 5. Une page mise en page avec des tableaux est illisible pour un lecteur d'écran et impossible à rendre responsive.

7 Valider son HTML

Le navigateur est très tolérant : il affiche une page même si des balises sont mal fermées, en devinant ce qu'on a voulu écrire. Cette tolérance est un piège — chaque navigateur devine à sa manière, et le jour où l'affichage se dérègle, l'erreur est déjà enfouie sous cent lignes de code.

Le validateur du W3C

Le service **validator.w3.org** vérifie qu'une page respecte la spécification HTML5. Il propose trois modes : par **adresse** (page en ligne), par **fichier** téléversé, ou par **copier-coller** du code — c'est ce dernier mode que tu utiliseras en TP.

Lire un message d'erreur

Chaque message du validateur comporte trois parties : la **ligne et la colonne** concernées, la **description** de l'erreur, et un **extrait** du code fautif avec la position surlignée.

Exemple de message

```

Error: End tag section seen, but there were open elements.
From line 24, column 3; to line 24, column 12

```

Traduction : à la ligne 24, on ferme une `<section>` alors qu'un élément ouvert à l'intérieur ne l'a pas été. Il manque une balise fermante quelque part au-dessus.

Message fréquent	Ce qu'il faut corriger
End tag X seen, but there were open elements	Une balise ouverte à l'intérieur n'a pas été fermée
Unclosed element X	La balise X n'est jamais fermée
Duplicate ID "xyz"	Deux éléments portent le même id
An img element must have an alt attribute	Une image sans alt
Element X not allowed as child of element Y	Mauvaise imbrication (un <code><p></code> directement dans un <code></code> , par exemple)
Consider adding a lang attribute	Il manque <code>lang="fr"</code> sur la balise <code><html></code>
Stray end tag X	Une balise fermante sans ouvrante correspondante

Méthode — corriger de haut en bas

Une seule balise oubliée peut produire dix messages d'erreur en cascade. Corrige donc **la première erreur de la liste**, puis relance la validation : le nombre d'erreurs chute souvent d'un coup.

Distingue aussi les **Errors** (rouge), qui doivent toutes disparaître, des **Warnings** (jaune), qui signalent une maladresse sans invalider la page.

Prévenir plutôt que corriger

Trois habitudes suppriment la majorité des erreurs avant même la validation :

- **indenter** systématiquement (Maj+Alt+F dans VS Code, avec Prettier) ;
- écrire la balise **fermante immédiatement** après l'ouvrante, puis remplir le contenu ;
- se fier au **soulignement rouge** de VS Code, qui repère déjà une partie des erreurs.

8 Lexique de la semaine

Terme	Définition
Balise (<i>tag</i>)	Marqueur qui décrit un morceau de contenu : <code><p></code> , <code></p></code>
Élément	L'ensemble balise ouvrante + contenu + balise fermante
Attribut	Information complémentaire dans une balise ouvrante (<code>nom="valeur"</code>)
Auto-fermant	Balise sans contenu ni fermeture : <code></code> , <code>
</code> , <code><meta></code>
Imbrication	Placement d'un élément dans un autre ; les balises se ferment dans l'ordre inverse
Sémantique	Choix d'une balise selon le <i>sens</i> du contenu, pas son apparence
<code><head></code>	Zone des métadonnées, invisible pour le visiteur
<code><body></code>	Zone du contenu visible de la page
id	Identifiant unique d'un élément dans la page
Bloc / en ligne	Un bloc occupe toute la largeur ; un élément en ligne reste dans le fil du texte
Chemin relatif	Chemin exprimé depuis le fichier courant (<code>images/logo.png</code>)
alt	Texte alternatif décrivant une image
Validation	Vérification de la conformité d'une page via <code>validator.w3.org</code>

Résumé de la semaine

Ce qu'il faut retenir

- HTML est un langage de **balisage** : il décrit la structure, pas l'apparence
- Les balises s'**imbriquent** comme des parenthèses et forment un **arbre**
- Toute page suit le même squelette : `<!DOCTYPE>`, `<html lang>`, `<head>`, `<body>`
- Le `<head>` contient les métadonnées invisibles ; le `<body>`, tout le visible
- Les balises **sémantiques** (`<header>`, `<nav>`, `<main>`, `<article>`, `<footer>`) décrivent le rôle de chaque zone
- **Un seul** `<h1>` par page, et pas de saut de niveau dans les titres
- Un `id` est **unique** ; un `alt` est **obligatoire** sur chaque image
- Les tableaux servent à des **données**, jamais à la mise en page
- Une page se **valide** sur `validator.w3.org`, et les erreurs se corrigent de haut en bas

Fil rouge — MiniForum : structure HTML du forum

Les deux pages du MiniForum prennent forme en HTML pur : la page d'accueil avec sa liste de sujets, et la page de détail d'un sujet avec ses messages codés en dur.

Aucun style cette semaine : c'est voulu. La structure doit tenir debout toute seule. C'est aussi le meilleur moment pour vérifier qu'elle est correcte — une fois le CSS posé, plus personne ne regarde le HTML.

Pour la semaine 3

- Les deux pages HTML du MiniForum terminées et poussées sur GitHub
- Chacune passe `validator.w3.org` **sans erreur**
- La semaine 3 ajoutera les **formulaires** et les bases de l'**accessibilité** : c'est encore du HTML, et ce sera la dernière semaine avant le CSS

Le détail pas à pas se trouve dans la fiche *S02_TP*.